

Clase 151 — Rootkits y bootkits

Parte: **6 — Análisis de malware** · Fuente: *The Art of Memory Forensics* (Ligh et al.) y *Windows Internals*
 Duración estimada: **130 min** · Nivel: **Experto**

Objetivo

Estudiar el malware que busca el sigilo máximo: rootkits (usuario y kernel) y bootkits que se cargan antes del sistema operativo. El alumno aprenderá las técnicas de ocultación (hooking, DKOM, filtros), por qué el análisis de memoria es imprescindible para detectarlos, y cómo el arranque seguro y las protecciones modernas cambian el panorama.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Distinguir** rootkits de usuario, de kernel y bootkits por su nivel de privilegio.
2. **Explicar** técnicas de ocultación: hooking (IAT/inline/SSDT), DKOM y filtros.
3. **Detectar** procesos, DLLs y conexiones ocultas con forense de memoria.
4. **Describir** el arranque (MBR/UEFI) y cómo un bootkit lo subvierte.
5. **Valorar** el papel de Secure Boot, PatchGuard y DSE como defensas.

Temas

#	Tema	Por qué importa
1	Anillos de privilegio y superficie kernel	Define el poder del rootkit
2	Rootkits de usuario (API hooking)	Ocultación básica en user-land
3	Rootkits de kernel (SSDT, DKOM)	Ocultan a nivel del propio SO
4	Bootkits (MBR/VBR/UEFI)	Persisten antes del SO
5	Forense de memoria para detección	Los rootkits mienten al SO en vivo
6	Defensas: Secure Boot, PatchGuard, DSE, HVCI	Elevan la barrera
7	Firma de drivers y BYOVD	Vector actual de kernel

Definiciones y características

- **Rootkit:** malware que oculta su presencia al sistema. Característica clave: **sigilo estructural**.
- **DKOM (Direct Kernel Object Manipulation):** altera estructuras del kernel (p. ej. desenlaza un `EPROCESS`). Característica clave: **oculta procesos** sin hooks.
- **SSDT hooking:** intercepta llamadas del sistema. Característica clave: **control de syscalls**.
- **Bootkit:** infecta MBR/VBR/UEFI para cargarse antes del SO. Característica clave: **persistencia pre-SO**.

- **BYOVD (Bring Your Own Vulnerable Driver):** cargar un driver firmado vulnerable para entrar al kernel. Característica clave: **evade la firma obligatoria**.

Herramientas y preparación

- **Volatility 3** (`malfind`, `ssdt`, `driverscan`, `psxview`, `netscan`).
- **GMER/RootkitRevealer** (conceptual) y comparación cross-view.
- **PE-bear/Ghidra** para drivers `.sys`.
- Referencias de arranque: **UEFI spec**, documentación de Secure Boot.

 **Nota ética y de seguridad:** los rootkits de kernel pueden dañar el SO invitado. Trabaja en la VM aislada con snapshot y asume que la VM quedará inservible tras la ejecución. Nunca cargues drivers maliciosos en un host real. El estudio es para **detección y defensa**.

Laboratorio guiado

1. Restaura `base-clean`. Captura un volcado de memoria de referencia (limpio) para comparar.
2. Detona una muestra con capacidad de ocultación (o usa un caso de estudio con memoria provista) y captura RAM.
3. Con Volatility, ejecuta `windows.psxview`: compara las distintas fuentes de procesos; un proceso presente en una lista y ausente en otra sugiere **DKOM**.
4. Revisa `windows.ssdt` para entradas que no apunten a `ntoskrnl / win32k` (posible **SSDT hook**).
5. Usa `windows.driverscan / modules` para hallar drivers no firmados o sospechosos; extrae el `.sys` y analízalo en Ghidra.
6. Con `windows.netscan`, busca conexiones que las herramientas en vivo no mostraban (ocultación de red).
7. Si es un bootkit, examina el MBR/VBR o la partición EFI en busca de modificaciones; documenta el vector de arranque.
8. Redacta hallazgos: técnica de ocultación, evidencia cross-view y cómo un EDR/forense lo detectaría. Descarta la VM.

Ejercicios

1. Explica la diferencia práctica entre un hook inline y DKOM.
2. Interpreta un `psxview` con discrepancias y concluye qué está oculto.
3. Analiza un driver `.sys` y localiza su rutina de ocultación.
4. Describe cómo Secure Boot frena los bootkits UEFI.
5. Explica BYOVD y por qué sigue funcionando pese a la firma obligatoria.
6. Diseña una detección de carga de drivers vulnerables conocidos.

Reto verificable

Entrega un **informe de rootkit** identificando la técnica de ocultación con evidencia de memoria (cross-view/SSDT/driverscan), el artefacto oculto (proceso/conexión/driver) y una recomendación de detección.

Criterio de aceptación: el informe muestra al menos una discrepancia cross-view o un hook/driver anómalo con la salida de Volatility que lo respalda, y explica por qué el análisis en disco/vivo no bastaba.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Herramientas en vivo "no ven nada"	El rootkit las engaña; usa forense de memoria offline
VM se cuelga al cargar el driver	Comportamiento esperado en kernel; trabaja con snapshot
<code>ssdt</code> limpio pero algo se oculta	Puede ser DKOM o hooks inline; combina varias vistas
Driver sospechoso pero firmado	Posible BYOVD; verifica CVE del driver, no solo la firma
No hallo el bootkit	Revisa MBR/VBR y partición EFI, no solo el sistema de archivos

? Preguntas frecuentes

? **¿Por qué memoria y no disco?** Porque el rootkit manipula el SO en ejecución; la memoria conserva la evidencia real que las APIs comprometidas ocultan.

? **¿Siguen siendo relevantes los bootkits?** Sí, especialmente UEFI. Secure Boot ayuda, pero han aparecido bootkits que lo evaden con vulnerabilidades del firmware.

? **¿Cómo entran hoy al kernel?** Con frecuencia vía BYOVD: cargan un driver firmado pero vulnerable para obtener ejecución en kernel sin necesitar su propio driver firmado.

🔗 Referencias

- Ligh et al., *The Art of Memory Forensics*.
- Yosifovich et al., *Windows Internals* (arranque y kernel).
- Volatility 3. <https://github.com/volatilityfoundation/volatility3>
- MITRE ATT&CK — Rootkit (T1014). <https://attack.mitre.org/techniques/T1014/>

➡ Siguiente clase

Clase 152 - Analisis de documentos maliciosos: macros y PDF